



B.Tech. CSE VI CSS 336
ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING LAB

Name: Samina Parveen

Roll Number: BTECH/10007/23

Assignment- 4

1. Assume that we want to control the opening of a gate by changing the amount of voltage we pass to the gate's electronic mechanism. If the gate can open from 0 to 90 degrees only, use a potentiometer to control the angle of opening of the gate. Use a LCD display to display by what angle the gate has currently been opened.

```
|  
|— main.py  
|— lcd_api.py  
|— i2c_lcd.py  
|— diagram.json
```

main.py

```
from machine import Pin, ADC, PWM, I2C  
import time  
from i2c_lcd import I2cLcd  
  
# potentiometer  
pot = ADC(26)  
  
# servo motor  
servo = PWM(Pin(15))  
servo.freq(50)  
  
# LCD setup  
i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)  
lcd = I2cLcd(i2c, 0x27, 2, 16)  
  
def set_servo_angle(angle):  
    duty = int(1638 + (angle / 180) * 6553)  
    servo.duty_u16(duty)  
  
while True:  
  
    value = pot.read_u16()  
  
    angle = int((value / 65535) * 90)
```

```
set_servo_angle(angle)

lcd.clear()
lcd.putstr("Gate Angle:")
lcd.move_to(0,1)
lcd.putstr(str(angle) + " deg")

time.sleep(0.5)
```

lcd_api.py

```
import time

class LcdApi:

    LCD_CLR = 0x01
    LCD_HOME = 0x02

    def __init__(self, num_lines, num_columns):
        self.num_lines = num_lines
        self.num_columns = num_columns

    def clear(self):
        self.hal_write_command(self.LCD_CLR)
        time.sleep_ms(2)

    def move_to(self, col, row):
        addr = col + 0x40 * row
        self.hal_write_command(0x80 | addr)

    def putchar(self, char):
        self.hal_write_data(ord(char))

    def putstr(self, string):
        for char in string:
            self.putchar(char)
```

i2c_lcd.py

```
from lcd_api import LcdApi
from machine import I2C
```

```

import time

class I2cLcd(LcdApi):

    def __init__(self, i2c, i2c_addr, num_lines, num_columns):
        self.i2c = i2c
        self.i2c_addr = i2c_addr
        super().__init__(num_lines, num_columns)

    def hal_write_command(self, cmd):
        self.i2c.writeto(self.i2c_addr, bytes([cmd]))
        time.sleep_ms(1)

    def hal_write_data(self, data):
        self.i2c.writeto(self.i2c_addr, bytes([data]))
        time.sleep_ms(1)

```

diagram.json (Wokwi Circuit)

```

{
  "version": 1,
  "author": "Samina",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-pi-pico", "id": "pico", "top": 0, "left": 0 },

    { "type": "wokwi-potentiometer", "id": "pot", "top": 160, "left": -80 },

    { "type": "wokwi-servo", "id": "servo", "top": -120, "left": 200 },

    { "type": "wokwi-lcd1602", "id": "lcd", "top": -160, "left": -220,
      "attrs": { "pins": "i2c" }
    }
  ],

  "connections": [

    [ "pot:VCC", "pico:3V3", "red" ],
    [ "pot:GND", "pico:GND", "black" ],
    [ "pot:SIG", "pico:GP26", "green" ],

    [ "servo:PWM", "pico:GP15", "green" ],
    [ "servo:V+", "pico:VBUS", "red" ],

```

```
[ "servo:GND", "pico:GND", "black" ],

[ "lcd:SDA", "pico:GP0", "green" ],
[ "lcd:SCL", "pico:GP1", "green" ],
[ "lcd:VCC", "pico:3V3", "red" ],
[ "lcd:GND", "pico:GND", "black" ]
]
}
```

2. Assume you are the security incharge of a nuclear reactor. One of your responsibilities includes monitoring the temperature of the reactor. The optimal temperature of the reactor is between 10 degree Celsius to 33 degree celsius. Your dashboard should blink a green LED as long as the temperature is in the optimal range. In case the temperature moves out of the optimal range you should start flashing a yellow led. As soon as the system has spent more than 5 seconds outside of the optimal zone you should start flashing a bank of three red LEDs. You should also flash a Danger signal on a LCD screen in case of such an eventuality. The temperature in celsius is obtained using the following formula

$$temp_celsius = \frac{1}{\log(\frac{1}{\frac{65535}{x} - 1}) / \beta + \frac{1}{298.15}} - 273.15 \quad (1)$$

main.py

```
from machine import Pin, ADC, I2C
import time
import math
from i2c_lcd import I2cLcd

# thermistor
therm = ADC(26)

# LEDs
green = Pin(15, Pin.OUT)
yellow = Pin(16, Pin.OUT)
red1 = Pin(17, Pin.OUT)
red2 = Pin(18, Pin.OUT)
red3 = Pin(19, Pin.OUT)

# LCD
i2c = I2C(0, scl=Pin(1), sda=Pin(0))
lcd = I2cLcd(i2c, 0x27, 2, 16)
```

```
beta = 3950
```

```
out_time = 0
```

```
while True:
```

```
    x = therm.read_u16()
```

```
    # temperature formula
```

```
    temp = (1/(math.log((1/65535*x)-1)/beta + 1/298.15))-273.15
```

```
    lcd.clear()
```

```
    lcd.putstr("Temp:")
```

```
    lcd.move_to(0,1)
```

```
    lcd.putstr(str(round(temp,2))+ " C")
```

```
    # Optimal range
```

```
    if 10 <= temp <= 33:
```

```
        green.toggle()
```

```
        yellow.off()
```

```
        red1.off()
```

```
        red2.off()
```

```
        red3.off()
```

```
        out_time = 0
```

```
    else:
```

```
        green.off()
```

```
        yellow.toggle()
```

```
        out_time += 1
```

```
        if out_time > 5:
```

```
            red1.toggle()
```

```
            red2.toggle()
```

```
            red3.toggle()
```

```
            lcd.clear()
```

```
            lcd.putstr("!!! DANGER !!!")
```

```
time.sleep(1)
```

lcd_api.py

```
import time
```

```
class LcdApi:
```

```
    LCD_CLR = 0x01
```

```
    LCD_HOME = 0x02
```

```
    def __init__(self, num_lines, num_columns):
```

```
        self.num_lines = num_lines
```

```
        self.num_columns = num_columns
```

```
    def clear(self):
```

```
        self.hal_write_command(self.LCD_CLR)
```

```
        time.sleep_ms(2)
```

```
    def move_to(self, col, row):
```

```
        addr = col + 0x40 * row
```

```
        self.hal_write_command(0x80 | addr)
```

```
    def putchar(self, char):
```

```
        self.hal_write_data(ord(char))
```

```
    def putstr(self, string):
```

```
        for char in string:
```

```
            self.putchar(char)
```

i2c_lcd.py

```
from lcd_api import LcdApi
```

```
import time
```

```
class I2cLcd(LcdApi):
```

```
    def __init__(self, i2c, i2c_addr, num_lines, num_columns):
```

```
        self.i2c = i2c
```

```
        self.i2c_addr = i2c_addr
```

```
        super().__init__(num_lines, num_columns)
```

```
    def hal_write_command(self, cmd):
```

```
        self.i2c.writeto(self.i2c_addr, bytes([cmd]))
```

```
time.sleep_ms(1)
```

```
def hal_write_data(self, data):  
    self.i2c.writeto(self.i2c_addr, bytes([data]))  
    time.sleep_ms(1)
```

diagram.json (Wokwi Circuit)

```
{  
  "version": 1,  
  "author": "Samina",  
  "editor": "wokwi",  
  "parts": [  
    { "type": "wokwi-pi-pico", "id": "pico", "top": 0, "left": 0 },  
  
    { "type": "wokwi-thermistor", "id": "therm", "top": 150, "left": -120 },  
  
    { "type": "wokwi-led", "id": "green", "top": -100, "left": 200, "attrs": {"color": "green"} },  
    { "type": "wokwi-led", "id": "yellow", "top": -60, "left": 200, "attrs": {"color": "yellow"} },  
  
    { "type": "wokwi-led", "id": "red1", "top": -20, "left": 200, "attrs": {"color": "red"} },  
    { "type": "wokwi-led", "id": "red2", "top": 20, "left": 200, "attrs": {"color": "red"} },  
    { "type": "wokwi-led", "id": "red3", "top": 60, "left": 200, "attrs": {"color": "red"} },  
  
    { "type": "wokwi-lcd1602", "id": "lcd", "top": -180, "left": -240,  
      "attrs": { "pins": "i2c" }  
    },  
  ],  
  
  "connections": [  
  
    ["therm:VCC", "pico:3V3", "red"],  
    ["therm:GND", "pico:GND", "black"],  
    ["therm:SIG", "pico:GP26", "green"],  
  
    ["green:A", "pico:GP15", "green"],  
    ["green:C", "pico:GND", "black"],  
  
    ["yellow:A", "pico:GP16", "green"],  
    ["yellow:C", "pico:GND", "black"],  
  
    ["red1:A", "pico:GP17", "green"],  
    ["red1:C", "pico:GND", "black"],
```



```

["red2:A","pico:GP18","green"],
["red2:C","pico:GND","black"],

["red3:A","pico:GP19","green"],
["red3:C","pico:GND","black"],

["lcd:SDA","pico:GP0","green"],
["lcd:SCL","pico:GP1","green"],
["lcd:VCC","pico:3V3","red"],
["lcd:GND","pico:GND","black"]
]
}

```

3. Assume you are driving car and accelerating the car is dependent on the ambient lighting of the surrounding. You are currently driving at velocity v m/s and you are only allowed to accelerate if the ambient light doubles w.r.t to the initial value. Use a LCD display to display whether the car is allowed to accelerate at any instant in time.

project

```

|
├── main.py
├── lcd_api.py
├── i2c_lcd.py
└── diagram.json

```

main.py

```

from machine import Pin, ADC, I2C
import time
from i2c_lcd import I2cLcd

# LDR sensor
ldr = ADC(26)

# LCD
i2c = I2C(0, scl=Pin(1), sda=Pin(0))
lcd = I2cLcd(i2c, 0x27, 2, 16)

# initial velocity
velocity = 20

# read initial light
initial_light = ldr.read_u16()

```

```
while True:

    light = ldr.read_u16()

    lcd.clear()

    if light >= 2 * initial_light:
        lcd.putstr("ACCELERATION")
        lcd.move_to(0,1)
        lcd.putstr("ALLOWED")
    else:
        lcd.putstr("ACCELERATION")
        lcd.move_to(0,1)
        lcd.putstr("NOT ALLOWED")

    time.sleep(1)
```

lcd_api.py

```
import time

class LcdApi:

    LCD_CLR = 0x01
    LCD_HOME = 0x02

    def __init__(self, num_lines, num_columns):
        self.num_lines = num_lines
        self.num_columns = num_columns

    def clear(self):
        self.hal_write_command(self.LCD_CLR)
        time.sleep_ms(2)

    def move_to(self, col, row):
        addr = col + 0x40 * row
        self.hal_write_command(0x80 | addr)

    def putchar(self, char):
        self.hal_write_data(ord(char))

    def putstr(self, string):
        for char in string:
            self.putchar(char)
```

i2c_lcd.py

```
from lcd_api import LcdApi
import time
```

```
class I2cLcd(LcdApi):

    def __init__(self, i2c, i2c_addr, num_lines, num_columns):
        self.i2c = i2c
        self.i2c_addr = i2c_addr
        super().__init__(num_lines, num_columns)

    def hal_write_command(self, cmd):
        self.i2c.writeto(self.i2c_addr, bytes([cmd]))
        time.sleep_ms(1)

    def hal_write_data(self, data):
        self.i2c.writeto(self.i2c_addr, bytes([data]))
        time.sleep_ms(1)
```

diagram.json (Wokwi circuit)

```
{
  "version": 1,
  "author": "Samina",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-pi-pico", "id": "pico", "top": 0, "left": 0 },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr", "top": 150, "left": -120 },
    { "type": "wokwi-lcd1602", "id": "lcd", "top": -170, "left": -240,
      "attrs": { "pins": "i2c" }
    }
  ],
  "connections": [

    ["ldr:VCC", "pico:3V3", "red"],
    ["ldr:GND", "pico:GND", "black"],
    ["ldr:AO", "pico:GP26", "green"],

    ["lcd:SDA", "pico:GP0", "green"],
```

```

["lcd:SCL","pico:GP1","green"],
["lcd:VCC","pico:3V3","red"],
["lcd:GND","pico:GND","black"]
]
}

```

4. Extend the previous program by allowing the user to establish a steady state (conceptually similar to cruise control mode in real cars) by pressing a push button. The car is only allowed to accelerate from the current steady state by a factor of 1.2 if the ambient light doubles. The driver has the option of setting his steady state at any point in time.

```

project
├── main.py
├── lcd_api.py
├── i2c_lcd.py
└── diagram.json

```

main.py

```

from machine import Pin, ADC, I2C
import time
from i2c_lcd import I2cLcd

# sensors
ldr = ADC(26)

# push button
button = Pin(14, Pin.IN, Pin.PULL_DOWN)

# LCD
i2c = I2C(0, scl=Pin(1), sda=Pin(0))
lcd = I2cLcd(i2c, 0x27, 2, 16)

# initial velocity
velocity = 20
steady_state = velocity

# store initial light
initial_light = ldr.read_u16()

while True:

```

```

light = ldr.read_u16()

# set cruise control
if button.value() == 1:
    steady_state = velocity

# check light condition
if light >= 2 * initial_light:
    velocity = steady_state * 1.2
    state = "ACCELERATE"
else:
    velocity = steady_state
    state = "STEADY"

lcd.clear()
lcd.putstr(state)

lcd.move_to(0,1)
lcd.putstr("v=" + str(round(velocity,2)) + " m/s")

time.sleep(1)

```

lcd_api.py

```
import time
```

```
class LcdApi:
```

```
    LCD_CLR = 0x01
```

```
    LCD_HOME = 0x02
```

```
    def __init__(self, num_lines, num_columns):
```

```
        self.num_lines = num_lines
```

```
        self.num_columns = num_columns
```

```
    def clear(self):
```

```
        self.hal_write_command(self.LCD_CLR)
```

```
        time.sleep_ms(2)
```

```
    def move_to(self, col, row):
```

```
        addr = col + 0x40 * row
```

```
        self.hal_write_command(0x80 | addr)
```

```
    def putchar(self, char):
```

```
self.hal_write_data(ord(char))
```

```
def putstr(self, string):  
    for char in string:  
        self.putchar(char)
```

i2c_lcd.py

```
from lcd_api import LcdApi  
import time
```

```
class I2cLcd(LcdApi):
```

```
    def __init__(self, i2c, i2c_addr, num_lines, num_columns):  
        self.i2c = i2c  
        self.i2c_addr = i2c_addr  
        super().__init__(num_lines, num_columns)
```

```
    def hal_write_command(self, cmd):  
        self.i2c.writeto(self.i2c_addr, bytes([cmd]))  
        time.sleep_ms(1)
```

```
    def hal_write_data(self, data):  
        self.i2c.writeto(self.i2c_addr, bytes([data]))  
        time.sleep_ms(1)
```

diagram.json (Wokwi Circuit)

```
{  
  "version": 1,  
  "author": "Samina",  
  "editor": "wokwi",  
  "parts": [  
    { "type": "wokwi-pi-pico", "id": "pico", "top": 0, "left": 0 },  
  
    { "type": "wokwi-photoresistor-sensor", "id": "ldr", "top": 160, "left": -120 },  
  
    { "type": "wokwi-pushbutton", "id": "btn", "top": 100, "left": 200 },  
  
    { "type": "wokwi-lcd1602", "id": "lcd", "top": -170, "left": -240,  
      "attrs": { "pins": "i2c" }  
    }  
  ],  
}
```

```
"connections": [  
  
  ["ldr:VCC","pico:3V3","red"],  
  ["ldr:GND","pico:GND","black"],  
  ["ldr:AO","pico:GP26","green"],  
  
  ["btn:1","pico:GP14","green"],  
  ["btn:2","pico:GND","black"],  
  
  ["lcd:SDA","pico:GP0","green"],  
  ["lcd:SCL","pico:GP1","green"],  
  ["lcd:VCC","pico:3V3","red"],  
  ["lcd:GND","pico:GND","black"]  
]  
}
```